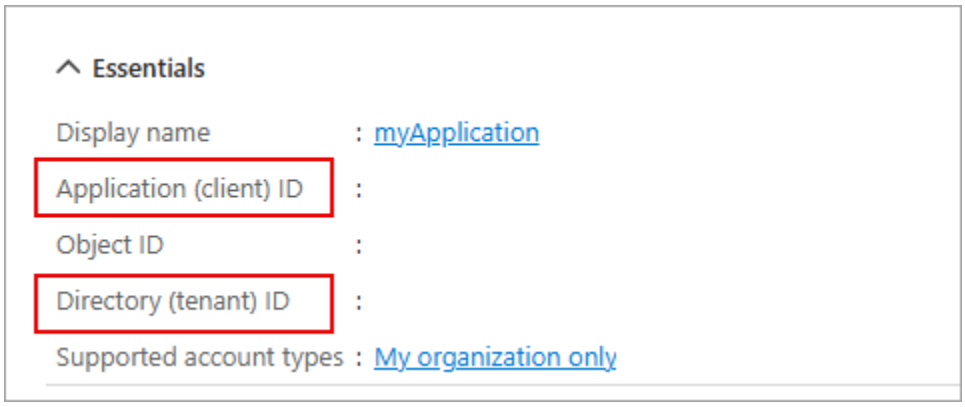
Implement interactive authentication with MSAL.NET

Register a new application

1. In your browser navigate to the Azure portal https://*portal.azure.com*; signing in with your Azure credentials if prompted.
2. In the portal, search for and select **App registrations**.
3. Select **+ New registration**, and when the **Register an application** page appears, enter your application's registration information:

Field	Value
Name	Enter myMsalApplication
Supported account types	Select **Accounts in this organizational directory only**
Redirect URI (optional)	Select **Public client/native (mobile & desktop)** and enter http://*localhost* in the box to the right.

4. Select **Register**. Microsoft Entra ID assigns a unique application (client) ID to your app, and you're taken to your application's **Overview** page.
5. In the **Essentials** section of the **Overview** page record the **Application (client) ID** and the **Directory (tenant) ID**. The information is needed for the application.



Create a .NET console app to acquire a token

Now that the needed resources are deployed to Azure the next step is to set up the console application. The following steps are performed in your local environment.

1. Create a folder named **authapp**, or a name of your choosing, for the project.
2. Launch **Visual Studio Code** and select **File > Open folder...** and select the project folder.

3. Select **View > Terminal** to open a terminal.
4. Run the following command in the VS Code terminal to create the .NET console application.

TypeCopy

dotnet new console

5. Run the following commands to add the **Microsoft.Identity.Client** and **dotenv.net** packages to the project.

TypeCopy

dotnet add package Microsoft.Identity.Client

dotnet add package dotenv.net

Configure the console application

In this section you create, and edit, a **.env** file to hold the secrets you recorded earlier.

1. Select **File > New file...** and create a file named *.env* in the project folder.
2. Open the **.env** file and add the following code. Replace **YOUR_CLIENT_ID**, and **YOUR_TENANT_ID** with the values you recorded earlier.

TypeCopy

CLIENT_ID="YOUR_CLIENT_ID"

TENANT_ID="YOUR_TENANT_ID"

3. Press **ctrl+s** to save your changes.

Add the starter code for the project

1. Open the *Program.cs* file and replace any existing contents with the following code. Be sure to review the comments in the code.

csharpTypeCopy

using Microsoft.Identity.Client;

using dotenv.net;

// Load environment variables from .env file

DotEnv.Load();

var envVars = DotEnv.Read();

// Retrieve Azure AD Application ID and tenant ID from environment variables

string _clientId = envVars["CLIENT_ID"];

string _tenantId = envVars["TENANT_ID"];

// ADD CODE TO DEFINE SCOPES AND CREATE CLIENT

// ADD CODE TO ACQUIRE AN ACCESS TOKEN

2. Press **ctrl+s** to save your changes.

Add code to complete the application

1. Locate the **// ADD CODE TO DEFINE SCOPES AND CREATE CLIENT** comment and add the following code directly after the comment. Be sure to review the comments in the code.

csharpTypeCopy

// Define the scopes required for authentication

string[] _scopes = { "User.Read" };

// Build the MSAL public client application with authority and redirect URI

var app = PublicClientApplicationBuilder.Create(_clientId)

.WithAuthority(AzureCloudInstance.AzurePublic, _tenantId)

.WithDefaultRedirectUri()

.Build();

2. Locate the **// ADD CODE TO ACQUIRE AN ACCESS TOKEN** comment and add the following code directly after the comment. Be sure to review the comments in the code.

csharpTypeCopy

// Attempt to acquire an access token silently or interactively

```
AuthenticationResult result;
try
{
   // Try to acquire token silently from cache for the first available account
   var accounts = await app.GetAccountsAsync();
   result = await app.AcquireTokenSilent(_scopes, accounts.FirstOrDefault())
            .ExecuteAsync();
}
catch (MsalUiRequiredException)
{
   // If silent token acquisition fails, prompt the user interactively
   result = await app.AcquireTokenInteractive(_scopes)
            .ExecuteAsync();
}

// Output the acquired access token to the console
Console.WriteLine($"Access Token:\n{result.AccessToken}");
```

3. Press **ctrl+s** to save the file, then **ctrl+q** to exit the editor.

Run the application

Now that the app is complete it's time to run it.

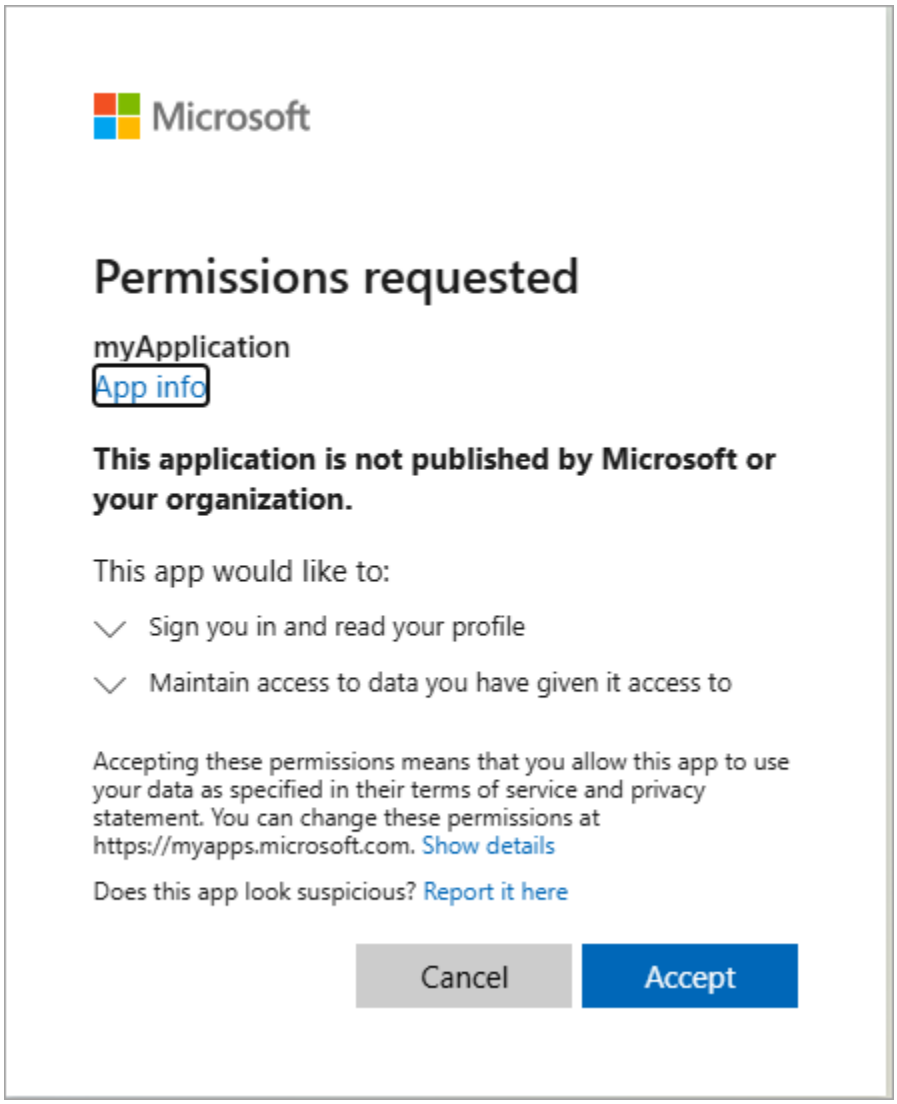
1. Start the application by running the following command:

TypeCopy

```
dotnet run
```

2. The app opens the default browser prompting you to select the account you want to authenticate with. If there are multiple accounts listed select the one associated with the tenant used in the app.
3. If this is the first time you've authenticated to the registered app you receive a **Permissions requested** notification asking you to approve the app to sign you

in and read your profile, and maintain access to data you have given it access to. Select **Accept**.



4. You should see the results similar to the example below in the console.

TypeCopy

Access Token:

eyJ0eXAiOiJKV1QiLCJub25jZSI6IlZF.....

5. Start the application a second time and notice you no longer receive the **Permissions requested** notification. The permission you granted earlier was cached. **Note:** If you have mutliple accounts, and With some account configurations, you might see the notification again.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. In your browser navigate to the Azure portal https*://portal.azure.com*; signing in with your Azure credentials if prompted.
2. Navigate to the resource group you created and view the contents of the resources used in this exercise.
3. On the toolbar, select **Delete resource group**.
4. Enter the resource group name and confirm that you want to delete it.

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.